信息隐藏技术 实验四

> 王峥 2211267 实验原理+秘密信息提取
>
> 李佳璐2211985 格式剖析
>
> 黄煜斐 2213028 秘密信息隐藏

一，实验原理

（一）音频信息隐藏技术

音频信息隐藏技术，是指在保证音频感知质量的前提下，将秘密信息嵌入音频载体中，并能在需要时无损地提取。它是数字水印和信息隐藏的重要分支之一，广泛应用于：

- 数字版权管理（DRM）
- 数据完整性验证
- 加密通信
- 音频指纹识别
- 认证/反盗用系统

该技术必须在“隐蔽性”和“鲁棒性”之间进行权衡，同时也要兼顾嵌入容量、计算效率等多种指标。

主要指标

1. 透明性（Imperceptibility）

嵌入后的音频在听感上必须和原音频几乎无差异，一般用 SNR、PESQ 或 MOS（主观评分）来评价。

2. 鲁棒性（Robustness）

嵌入的信息应能抵抗音频常见处理如：

- 有损压缩（如MP3、AAC）
- 添加噪声
- 重采样、剪切、混响等
- 传输错误

3. 嵌入容量（Capacity）

衡量单位时间内可嵌入信息的比特数，需兼顾不破坏音质的前提下尽可能提高嵌入量。

4. 安全性（Security）

防止恶意攻击者检测、修改或提取隐藏数据。常通过加密密钥、扩频等手段增强。

5. 不可检测性（Undetectability）

防止隐藏信息被**统计分析**或**机器学习检测模型**发现，是近年来研究热点。

常用隐藏域与技术原理

域	描述	常见算法
时域	直接对音频采样点操作	LSB、回声隐藏、时延编码
频域	使用FFT、DCT获取频谱嵌入	相位编码、频带编码、QIM
小波域（Wavelet）	多分辨率特性，适用于多尺度嵌入	DWT嵌入、小波包变换法
感知域	融合心理声学模型	MP3码本嵌入、MPEG Layer II嵌入
压缩域	在MP3、AAC等编码过程中直接嵌入	声码器隐藏、码本索引法

典型算法详解

LSB（Least Significant Bit）隐藏法

- 将信息嵌入采样值的最低有效位
- 简单高效，但鲁棒性差，一旦音频被处理就丢失信息

> 示例：
> 原采样点 `01100110`，隐藏比特为1，则修改后为 `01100111`

回声隐藏法（Echo Hiding）

- 向音频中添加延时的“回声”信号
- 延迟量或幅度差异代表不同比特值
- 优点：音质影响小、鲁棒性强

频域相位编码法

- 使用FFT变换，将信息编码进相位（如0表示0度，1表示180度）
- 因人耳对相位不敏感，透明性好

- 适合低速高安全场景

QIM（Quantization Index Modulation）量化调制法

- 根据信息对载体信号进行量化编码
- 可设计为盲检测（无需原音）
- 良好的鲁棒性，可对抗压缩与失真

DWT（离散小波变换）嵌入

- 把音频信号分解为多分辨率子带
- 在高频子带中嵌入数据
- 更符合人耳听觉模型，抗压缩性强

Spread Spectrum（扩频隐藏）

- 使用伪随机码将秘密信息扩频后嵌入
- 极强的抗干扰能力和安全性
- 通信安全领域使用较多

听觉模型与隐藏策略

音频信息隐藏依赖于**心理声学模型**来最大程度地隐匿数据。

1. 听觉阈值模型
- 利用人耳对频率响应曲线的不同灵敏度
- 高频、低频区域的阈值较高，可作为嵌入位置
2. 听觉掩蔽模型
- 同频率/时间邻近的强信号可掩蔽弱信号
- 在掩蔽效应区嵌入信息可最大限度地“藏起来”
3. 相位不敏感模型
- 人耳对相位变化不敏感，可对相位微调隐藏数据

（二）核心算法-LSB 算法

LSB 算法是音频信息隐藏中**最早被提出也是应用最广泛**的一种方法，它的基本思想是：

> 将待隐藏的秘密信息（如文本、图像、序列）**替换音频采样值的最低有效位**，因为这一位对音频整体波形影响极小，人耳几乎无法察觉。

LSB属于**时域隐藏技术**，操作简便，嵌入容量大，适用于**无压缩或轻压缩音频格式（如WAV）**。

基本原理

1.音频数字采样回顾：

假设一段WAV音频文件，每个采样点用16位二进制表示一个数值（常见为16-bit PCM编码），比如某一采样点：

`原始采样点值：1001 1010 1101 0110（二进制）`

最低有效位（LSB）是：最后一位 `0`

我们要嵌入的比特是： `1`
则替换后该采样点变为：

`修改后采样点值：1001 1010 1101 0111（二进制）`

从整体音频波形看，这种1位的变动几乎不改变原始音质。

---

2. 嵌入过程示意图（文字版）

```
音频采样数据：[10011010] [11001001] [01110011] …
待嵌入信息比特：[ 1 ] [ 0 ] [ 1 ]
LSB替换后音频：[10011011] [11001000] [01110011] …
```

---

3. 提取过程

提取时只需读取采样值的最低位，即可恢复秘密信息。例如：

```
已嵌入音频：[10011011] [11001000] [01110011]
提取LSB位：[    1     ] [    0     ] [    1     ]
```

LSB 算法的主要流程（伪代码）

嵌入过程

代码块

```
for (int i = 0; i < secret_bits.length; i++) {
    audio_samples[i] = (audio_samples[i] & 0xFE) | secret_bits[i];
}
```

解释：

- `& 0xFE`：将最低位清零
- `| secret_bits[i]`：将最低位设置为秘密比特

提取过程

代码块

```
for (int i = 0; i < num_bits_to_extract; i++) {
    secret_bits[i] = audio_samples[i] & 0x01;
}
```

实验过程

一、任选一种媒体文件，进行格式剖析(建议用UltraEdit)

> 通过 **UltraEdit**（十六进制编辑器）直接查看 WAV 文件的二进制结构，理解其文件格式。
>
> 验证 WAV 文件头中的关键参数（采样率、位深度、声道数等），并定位音频数据块。

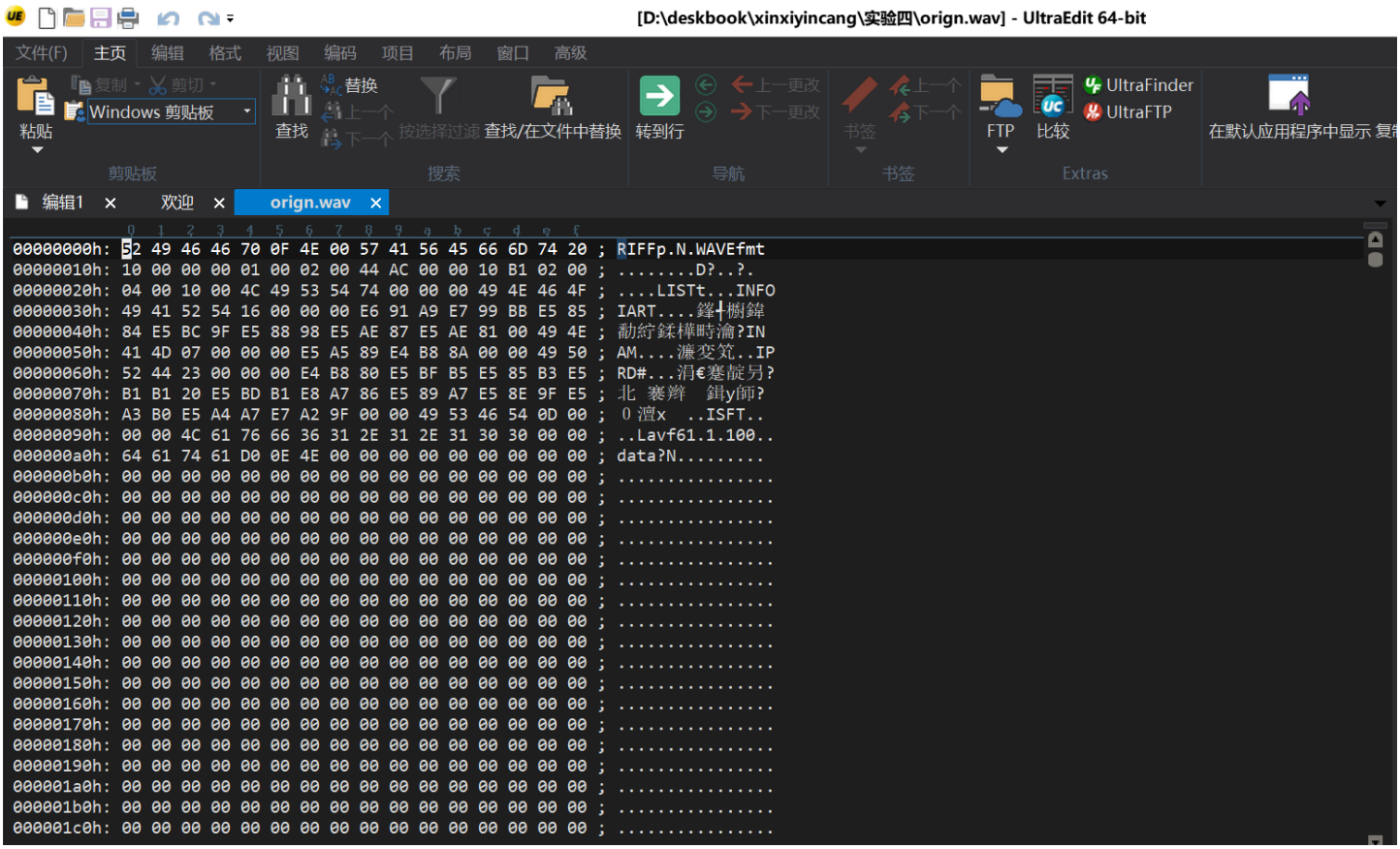
WAV 文件格式：基于 **RIFF（Resource Interchange File Format）** 结构，由多个“块（Chunk）”组成，每个块包含标识符、数据大小和具体内容。

关键块：

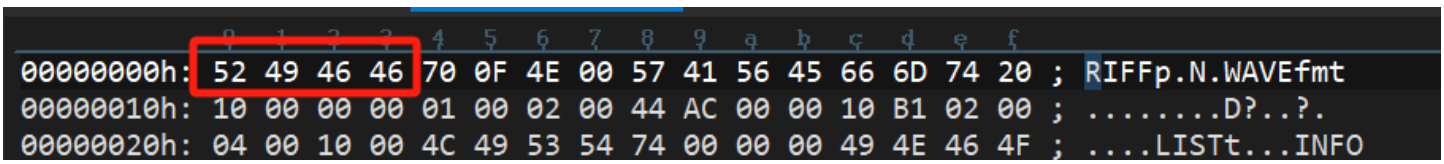
- **RIFF 头**：标识文件类型和总大小。
- **fmt 子块**：存储音频格式参数（如 PCM 编码、声道数、采样率）。
- **data 子块**：存储原始音频数据（PCM 样本）。

1. 关键字段解析示例

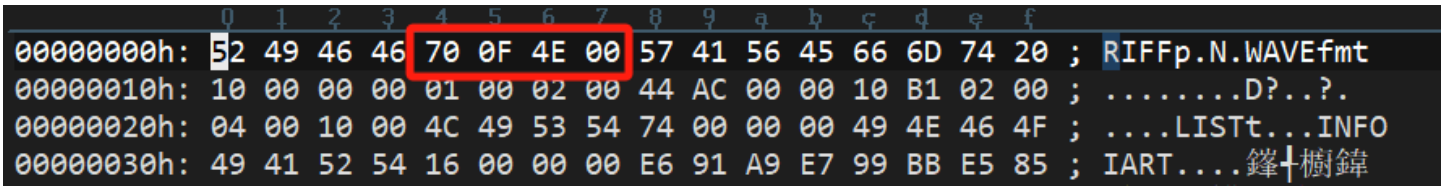
使用 UltraEdit 对 wav 文件进行格式剖析：



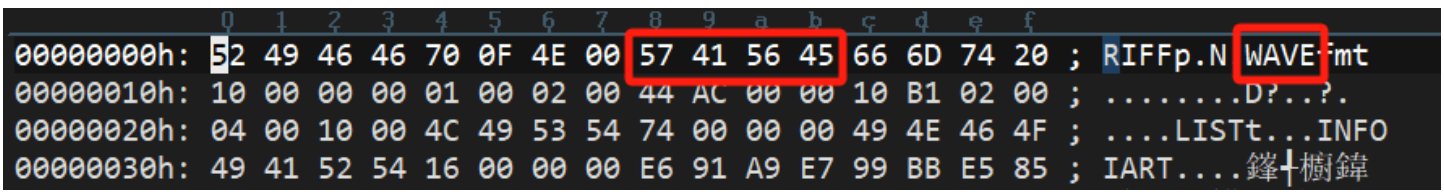
- wav文件开头四字节固定为 `RIFF` （ASCII 字符）



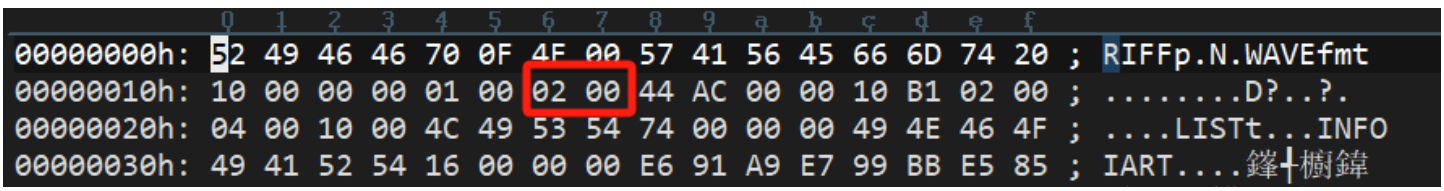
- 文件总大小（从 0x08 开始到文件末尾的字节数，小端存储）



- Format：固定为 WAVE（ASCII 字符）



- 声道数（2表示立体声）



- 采样率：44100 Hz（小端存储）

```
00000000h: 52 49 46 46 70 0F 4E 00 57 41 56 45 66 6D 74 20 ; RIFFp.N.WAVEfmt
00000010h: 10 00 00 00 01 00 02 00 44 AC 00 00 10 B1 02 00 ; ........D?..?.
00000020h: 04 00 10 00 4C 49 53 54 74 00 00 00 49 4E 46 4F ; ....LISTt...INFO
00000030h: 49 41 52 54 16 00 00 00 E6 91 A9 E7 99 BB E5 85 ; IART....鑝┼橱鍏
```

- 每秒字节数：

```
00000000h: 52 49 46 46 70 0F 4E 00 57 41 56 45 66 6D 74 20 ; RIFFp.N.WAVEfmt
00000010h: 10 00 00 00 01 00 02 00 44 AC 00 00 10 B1 02 00 ; ........D?..?.
00000020h: 04 00 10 00 4C 49 53 54 74 00 00 00 49 4E 46 4F ; ....LISTt...INFO
00000030h: 49 41 52 54 16 00 00 00 E6 91 A9 E7 99 BB E5 85 ; IART....鑝┼橱鍏
```

> 因实验文件由mp3文件转换的wav文件，会有一些额外字段
>
> `LIST` **块（元数据块）**
>
> - 存储 MP3 文件原有的元数据（如歌曲名、艺术家、专辑等）。
>
> `INFO` **子块**：包含键值对形式的元数据（如 `INAM` =歌曲名， `IART` =艺术家）。
>
> ```
> 00000010h: 10 00 00 00 01 00 02 00 44 AC 00 00 10 B1 02 00 ; ........D?..?.
> 00000020h: 04 00 10 00 4C 49 53 54 74 00 00 00 49 4E 46 4F ; ....LISTt...INFO
> 00000030h: 49 41 52 54 16 00 00 00 E6 91 A9 E7 99 BB E5 85 ; IART....鑝┼橱鍏
> 00000040h: 84 E5 BC 9F E5 88 98 E5 AE 87 E5 AE 81 00 49 4E ; 勐紵鍒樺畤瀹?IN
> ```

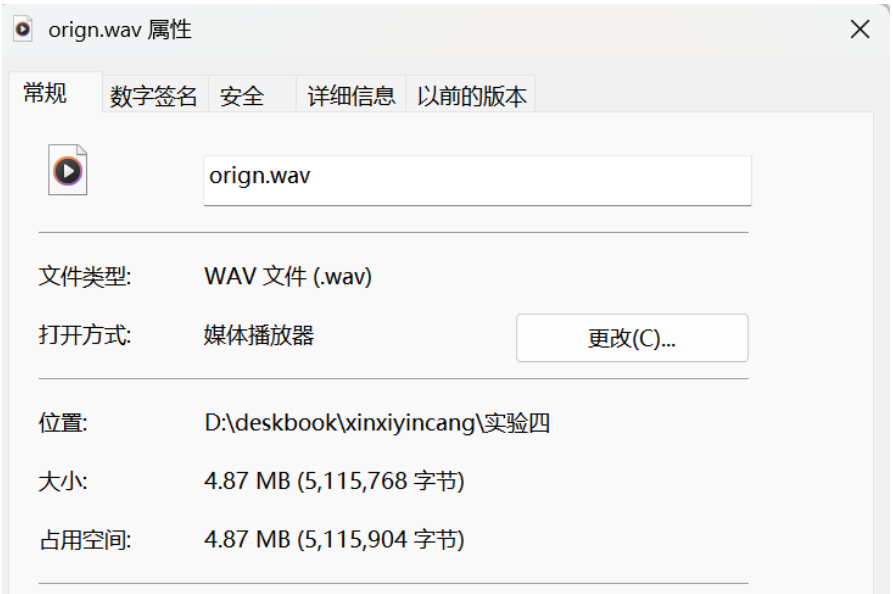
- data字段

`data` 块的起始位置不在 `0x24` ，而是被 `LIST` 等块挤到更后方

```
00000080h: A3 B0 E5 A4 A7 E7 A2 9F 00 00 49 53 46 54 0D 00 ; 0 澶x  ..ISFT..
00000090h: 00 00 4C 61 76 66 36 31 2E 31 2E 31 30 30 00 00 ; ..Lavf61.1.100..
000000a0h: 64 61 74 61 D0 0E 4E 00 00 00 00 00 00 00 00 00 ; data?N.........
```

- 音频数据的总字节数（小端存储）：5115678 字节

```
00000080h: A3 B0 E5 A4 A7 E7 A2 9F 00 00 49 53 46 54 0D 00 ; 0 澶x  ..ISFT..
00000090h: 00 00 4C 61 76 66 36 31 2E 31 2E 31 30 30 00 00 ; ..Lavf61.1.100..
000000a0h: 64 61 74 61 D0 0E 4E 00 00 00 00 00 00 00 00 00 ; data?N.........
```



2. 总结一般 WAV 文件基本结构

偏移地址	字段名	大小（字节）	说明
0x00~0x03	ChunkID	4	固定为 RIFF（ASCII 字符）
0x04~0x07	ChunkSize	4	文件总大小（从 0x08 开始到文件末尾的字节数，小端存储）
0x08~0x0B	Format	4	固定为 WAVE（ASCII 字符）
0x0C~0x0F	Subchunk1ID	4	fmt （包含末尾空格，ASCII 字符）
0x10~0x13	Subchunk1Size	4	fmt 块数据大小（通常为 16/18/40，取决于编码格式，小端存储）
0x14~0x15	AudioFormat	2	音频格式（1 表示 PCM，其他如 3 表示 IEEE Float，小端存储）
0x16~0x17	NumChannels	2	声道数（1=单声道，2=立体声，小端存储）
0x18~0x1B	SampleRate	4	采样率（如 44100 Hz，小端存储）
0x1C~0x1F	ByteRate	4	每秒字节数 = SampleRate × NumChannels × BitsPerSample/8（小端存储）
0x20~0x21	BlockAlign	2	每个样本的字节数 = NumChannels × BitsPerSample/8（小端存储）
0x22~0x23	BitsPerSample	2	位深度（如 16-bit，小端存储）
0x24~...	Subchunk2ID	4	data（ASCII 字符）
0x28~0x2B	Subchunk2Size	4	音频数据的总字节数（小端存储）
0x2C~...	Data	可变	原始音频数据（PCM 样本或其他编码数据）

二、秘密信息隐藏

2.1wav格式隐藏可能位置

WAV 文件由文件头和音频数据两部分组成，以下是一些**可能的信息隐藏位置**：

1. **文件头：**
   a. RIFF 头：包含文件类型和文件大小等信息。可以通过修改一些无关紧要的位来隐藏少量信息，例如在文件大小字段的低位字节中进行修改，因为这些修改可能不会影响文件的正常播放。
   b. 格式块：描述音频数据的格式，如采样率、声道数、位深度等。可以在一些保留字段或者对音频质量影响较小的参数位上进行信息嵌入。例如，在一些未使用的标志位或者采样率的低位字节进行修改。
   c. 扩展块：如果存在扩展块，这里面也可以寻找合适的位置进行信息隐藏。例如，在扩展块的一些自定义字段或者未被充分利用的字段中嵌入信息。
2. **音频数据：**
   a. 最低有效位（LSB）：音频数据通常以一定的位深度表示，如 8 位、16 位、32 位等。可以将隐藏信息嵌入到音频样本的最低有效位中。由于人耳对音频信号的微小变化不太敏感，这种方式可以在不明显影响音频质量的情况下隐藏大量信息。
   b. 音频帧间隙：在音频数据的帧之间，可能存在一些间隙或者未被充分利用的空间。可以在这些位置插入隐藏信息，通过特殊的编码方式来标识信息的起始和结束。
   c. 静音段：音频文件中的静音部分，即音频样本值接近零的区域，可以用来隐藏信息。可以通过对静音段的一些特征进行微小调整来编码隐藏信息，而不会对听觉效果产生明显影响。
   d. 频率域：通过对音频信号进行傅里叶变换，将其转换到频率域。在频率域中，可以选择一些对听觉感知影响较小的频率分量来隐藏信息，例如在高频部分或者一些非关键频率点上进行信息嵌入。

2.2秘密信息隐藏方法

常见方法有以下几种：

1. **最低有效位（LSB）替换法**

   WAV 文件中的音频数据通常以一定的位深度来表示，如 8 位、16 位或 32 位等。该方法是将秘密信息嵌入到音频样本的最低有效位中。由于人耳对音频信号的微小变化不敏感，只要修改的幅度足够小，就可以在不明显影响音频质量的情况下隐藏大量信息。假设音频样本值用 8 位二进制表示，原始样本值为 10101010，要嵌入的秘密信息为 1。则将最低位替换为 1，得到 10101011，实现了信息的隐藏。

2. 回声隐藏法

   利用人耳对回声的掩蔽效应，通过在原始音频信号中添加回声来隐藏秘密信息。回声的延迟时间和强度可以根据秘密信息进行编码。一般来说，较短的延迟时间和较弱的回声强度不易被人耳察觉，但可以被特定的解码器检测到并提取出隐藏的信息。若要隐藏信息 “0”，可以设置较短的回声延

迟时间和较低的回声强度；要隐藏信息 “1” ，则设置较长的延迟时间和较高的回声强度。通过这种方式将秘密信息编码到回声特征中。

3. 相位编码法

   对音频信号的相位信息进行调制来隐藏秘密信息。音频信号的相位在人耳感知声音的位置和方向等方面起着重要作用，但人耳对相位的微小变化相对不敏感。通过改变音频信号某些部分的相位来编码秘密信息，而不会对音频的整体感知产生太大影响。可以规定相位偏移 0° 表示信息 “0” ，相位偏移 180° 表示信息 “1” 。在音频的特定时间段内，根据要隐藏的信息对相位进行相应的调整，从而实现信息隐藏。

4. 变换域隐藏法

   先将音频信号从时域转换到变换域，如离散余弦变换（DCT）域、离散小波变换（DWT）域或傅里叶变换（FFT）域等。然后在变换域中选择一些对听觉感知影响较小的系数进行修改，以嵌入秘密信息。最后再将修改后的变换域系数转换回时域，得到隐藏信息后的音频信号。在 DCT 域中，对于一些高频系数，由于人耳对高频部分的变化相对不敏感，可以对这些系数进行适当的修改来编码秘密信息。修改完成后，通过逆 DCT 变换将系数转换回时域，得到隐藏信息后的音频文件。

5. 统计方法

   通过改变音频数据的统计特性来隐藏秘密信息。例如，调整音频样本的均值、方差或其他统计参数，使其在一定程度上符合预先设定的秘密信息编码规则。接收方可以通过分析音频数据的统计特性来提取隐藏的信息。可以规定音频样本均值的微小增加表示信息 “0” ，均值的微小减少表示信息 “1” 。通过对音频数据进行分组，并对每组数据的均值进行微调来嵌入秘密信息。接收方在接收到音频文件后，按照相同的分组方式计算均值，并根据均值的变化来提取秘密信息。

写根据秘密信息嵌入域不同，可以分为时域、变换域和压缩域方法，再根据所采用的技术将其进行详细划分，优缺点如下：

分类	方法	优点	缺点
时域方法	LSB隐写方法	1. 算法简单，实现容易2. 对音频质量影响较小，不易被听觉察觉3. 嵌入容量相对较大	1. 安全性较低，容易被统计分析等常规检测手段发现2. 抗攻击能力弱，如受到噪声干扰、重采样等操作时，隐藏信息易丢失
时域方法	相位编码方法	1. 利用人耳对相位变化相对不敏感的特性，隐蔽性较好 2. 可以在一定程度上抵抗一些简单的信号处理操作	1. 嵌入容量有限2. 编码和解码过程相对复杂，计算量较大3. 对相位变化的控制要求较高，否则可能影响音频质量
时域方法	回声隐藏方法	1. 隐蔽性较高，回声可利用人耳掩蔽效应不易察觉2. 相对来说有一定的抗攻	1. 嵌入信息的速率较低，可嵌入容量有限2. 回声参数设置不当可能会影响音

		击能力，如对一些常规的音频编辑操作有耐受性	频的听觉效果3. 检测和提取隐藏信息的算法较为复杂
时域方法	扩频方法	1. 具有较好的抗干扰和抗攻击能力，能在一定程度上抵抗噪声、滤波等操作2. 隐蔽性较好，隐藏信息不易被常规方法检测	1. 实现复杂度高，需要复杂的编码和解码算法2. 对音频信号的修改相对较大，可能会影响音频质量3. 嵌入容量相对有限
变换域方法	小波域变换方法	1. 多分辨率分析特性使其能更好地适应音频信号的特性，隐藏信息后音频质量损失较小2. 有较好的抗压缩和抗滤波能力，能在一定程度上抵抗常见的音频处理操作	1. 算法复杂度较高，计算量大2. 嵌入和提取隐藏信息的过程相对复杂3. 对参数设置要求较高，否则可能影响隐藏效果
变换域方法	离散余弦变换方法	1. 在图像和音频压缩领域应用广泛，技术相对成熟2. 能有效利用音频信号的频域特性，隐藏信息后音频质量相对稳定3. 有一定的抗噪声和抗干扰能力	1. 计算复杂度较高，尤其是在处理大数据量音频时2. 对DCT系数的修改可能会影响音频的压缩性能和重建质量3. 嵌入容量受限于DCT系数的可修改范围
变换域方法	离散傅里叶变换方法	1. 可以直观地在频域对音频信号进行操作，便于理解和实现 2. 对音频信号的频率特性分析较为准确，能合理选择隐藏信息的位置	1. 计算复杂度高，尤其是快速傅里叶变换（FFT）的计算量较大2. 隐藏信息后对音频信号的频域特性改变相对明显，可能影响音频质量3. 抗攻击能力有限，如遇到频域滤波等攻击时，隐藏信息易丢失
压缩域方法	MP3隐写方法	1. 直接在MP3压缩格式下操作，无需解压缩再嵌入信息，效率高2. 适应网络传输和存储需求，可在已压缩音频中隐藏信息	1. 受限于MP3压缩算法的特性，嵌入容量有限2. 不同的MP3编码参数可能影响隐藏效果和音频质量3. 安全性较低，容易被针对MP3格式的检测工具发现
压缩域方法	AAC隐写方法	1. AAC格式音频质量高，在高压缩比下仍能较好保持音频质量，隐写后音频质量损失相对小2. 与MP3类似，可直接在压缩域操	1. 算法依赖于AAC压缩标准，实现相对复杂2. 嵌入容量受AAC压缩机制限制，可嵌入信息量有限3. 对不同的AAC编码工具和

		作，方便在现有音频资源中隐藏信息	参数设置适应性需进一步优化
压缩域方法	VoIP隐写方法	1. 针对网络语音通信设计，能适应实时传输场景，在VoIP通信过程中隐藏信息具有实时性优势2. 可利用VoIP通信协议和编码特点，隐蔽性较好	1. 适用场景局限于VoIP通信，通用性较差2. 受VoIP网络环境和通信协议变化影响大，稳定性和兼容性面临挑战3. 隐藏信息的速率和容量需适应VoIP通信的实时性要求，相对有限

2.3LSB秘密信息隐藏实现

采用 LSB 隐藏法，将待隐藏的信息 payload 的每一个 bit 嵌入到 wav 音频文件的 Data 数据块的 Data 字段（以 BitsPerSample 位为一组）的 LSB（最低有效位）中，从而实现信息的隐藏。本次实验选取的待隐藏信息为随机生成序列，代码如下：

代码块

```
clear;
% 读取整个原文件数据
fid = fopen('E:\MATLAB\bin\orign.wav', 'r');
if fid == -1
    error('无法打开文件 orign.wav');
end
oa = fread(fid, '*uint8');
fclose(fid);

% 尝试按小端序查找'data'标识符
dataChunkPos_little = [];
for i = 1:length(oa) - 3
    if isequal(oa(i:i + 3), [100, 97, 116, 97])
        dataChunkPos_little = i;
        break;
    end
end
% 尝试按大端序查找'data'标识符
dataChunkPos_big = [];
oa_big = fliplr(reshape(oa, 4, [])); % 转换为大端序
oa_big = oa_big(:);
for i = 1:length(oa_big) - 3
    if isequal(oa_big(i:i + 3), [100, 97, 116, 97])
        dataChunkPos_big = i;
        break;
    end
end
```

```
if ~isempty(dataChunkPos_little)
    dataChunkPos = dataChunkPos_little;
elseif ~isempty(dataChunkPos_big)
    dataChunkPos = dataChunkPos_big;
else
    % 进一步尝试，将文件数据按字节逐个对比
    data_marker = [100, 97, 116, 97];
    for i = 1:length(oa) - 3
        found = true;
        for j = 0:3
            if oa(i + j) ~= data_marker(j + 1)
                found = false;
                break;
            end
        end
        if found
            dataChunkPos = i;
            break;
        end
    end
    if isempty(dataChunkPos)
        error('未找到data块，文件可能已损坏或格式不正确。');
    end
end

% data块的数据部分起始位置（跳过标识符和长度字段）
dataStart = dataChunkPos + 8;

% 计算可嵌入数据的长度
n = length(oa) - dataStart + 1;
d = randsrc(1, n, [1 0]);      % 生成随机二进制数据

% 嵌入数据到最低有效位
M = oa;
for i = 1:n
    M(dataStart + i - 1) = bitset(M(dataStart + i - 1), 1, d(i));
end

% 更新文件头中数据块的大小信息
% 已知音频数据总字节数（小端存储）为 5115678 字节，这里假设嵌入数据后大小不变（实际可根据嵌入情况调整）
data_size = 5115678;
data_size_bytes = typecast(uint32(data_size), 'uint8'); % 转换为字节表示
M(dataChunkPos + 4 : dataChunkPos + 7) = data_size_bytes; % 更新文件头中数据块大小字段

```

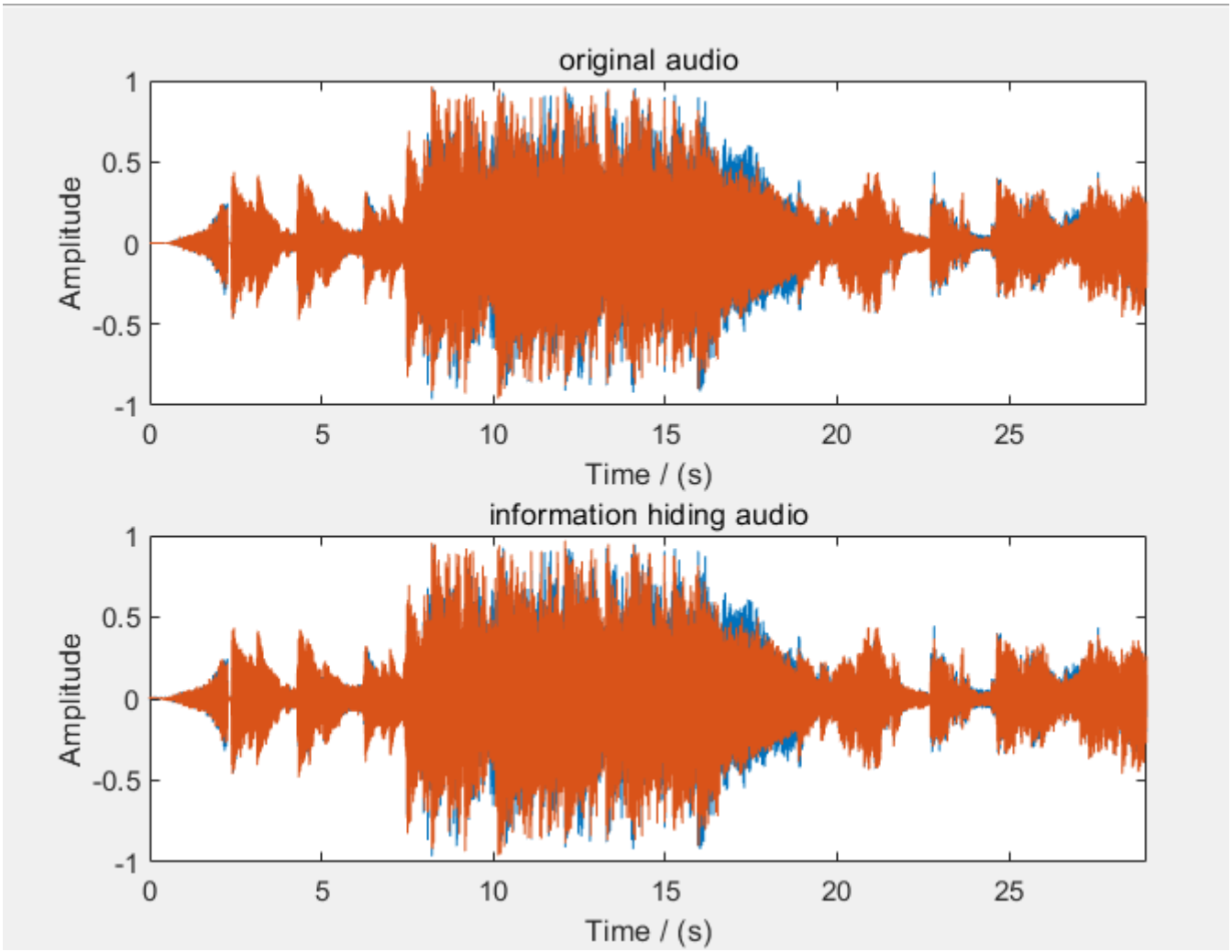
```
% 写入新文件
fid = fopen('D:\secret1.wav', 'wb');
if fid == -1
    error('无法创建文件 secret.wav');
end
fwrite(fid, M, 'uint8');
fclose(fid);

% 绘制波形图对比
figure;
[y1,Fs] = audioread('E:\MATLAB\bin\orign.wav');
x1 = (0:length(y1) - 1)/Fs;
subplot(2, 1, 1);
plot(x1, y1);
```

1. 读取音频文件：使用 `fopen` 函数以只读模式打开原始音频文件 `orign.wav`，若打开成功则通过 `fread` 函数读取文件的所有数据（以无符号 8 位整数形式），并存储在变量 `oa` 中，最后关闭文件。
2. 查找 `data` 块位置：
   - 先尝试按小端序在音频数据 `oa` 中查找 `data` 标识符（ASCII 码为 `[100, 97, 116, 97]`），若找到则记录其起始位置到 `dataChunkPos_little` 变量中。
   - 若小端序未找到，则将音频数据转换为大端序存储形式（通过 `reshape` 和 `fliplr` 函数），在转换后的数据中按大端序查找 `data` 标识符，若找到则记录其起始位置到 `dataChunkPos_big` 变量中。
   - 若小端序和大端序都未找到，则通过逐个字节对比的方式再次查找 `data` 标识符。若最终仍未找到，则抛出错误提示文件可能已损坏或格式不正确。
3. 确定数据嵌入位置和生成随机序列：根据找到的 `data` 标识符起始位置 `dataChunkPos`，计算 `data` 块中实际数据部分的起始位置 `dataStart`（跳过 `data` 标识符和其长度字段）。然后计算从 `data` 块数据起始位置到音频数据末尾的长度，作为可嵌入数据的长度 `n`，并使用 `randsrc` 函数生成一个长度为 `n` 的随机二进制序列 `d`。
4. 嵌入随机序列：将原始音频数据 `oa` 复制到变量 `M` 中，通过循环将随机二进制序列 `d` 逐位嵌入到 `M` 中从 `dataStart` 位置开始的每个字节的最低有效位。
5. 更新文件头信息：根据已知的音频数据总字节数（假设嵌入数据后大小不变），将其转换为 4 字节的无符号 8 位整数表示形式 `data_size_bytes`，然后更新音频数据 `M` 中文件头部分表示 `data` 块大小的字段。
6. 写入新音频文件：以二进制写入模式打开指定路径的新文件 `secret1.wav`，若打开成功则将修改后的音频数据 `M` 以无符号 8 位整数形式写入文件，最后关闭文件。

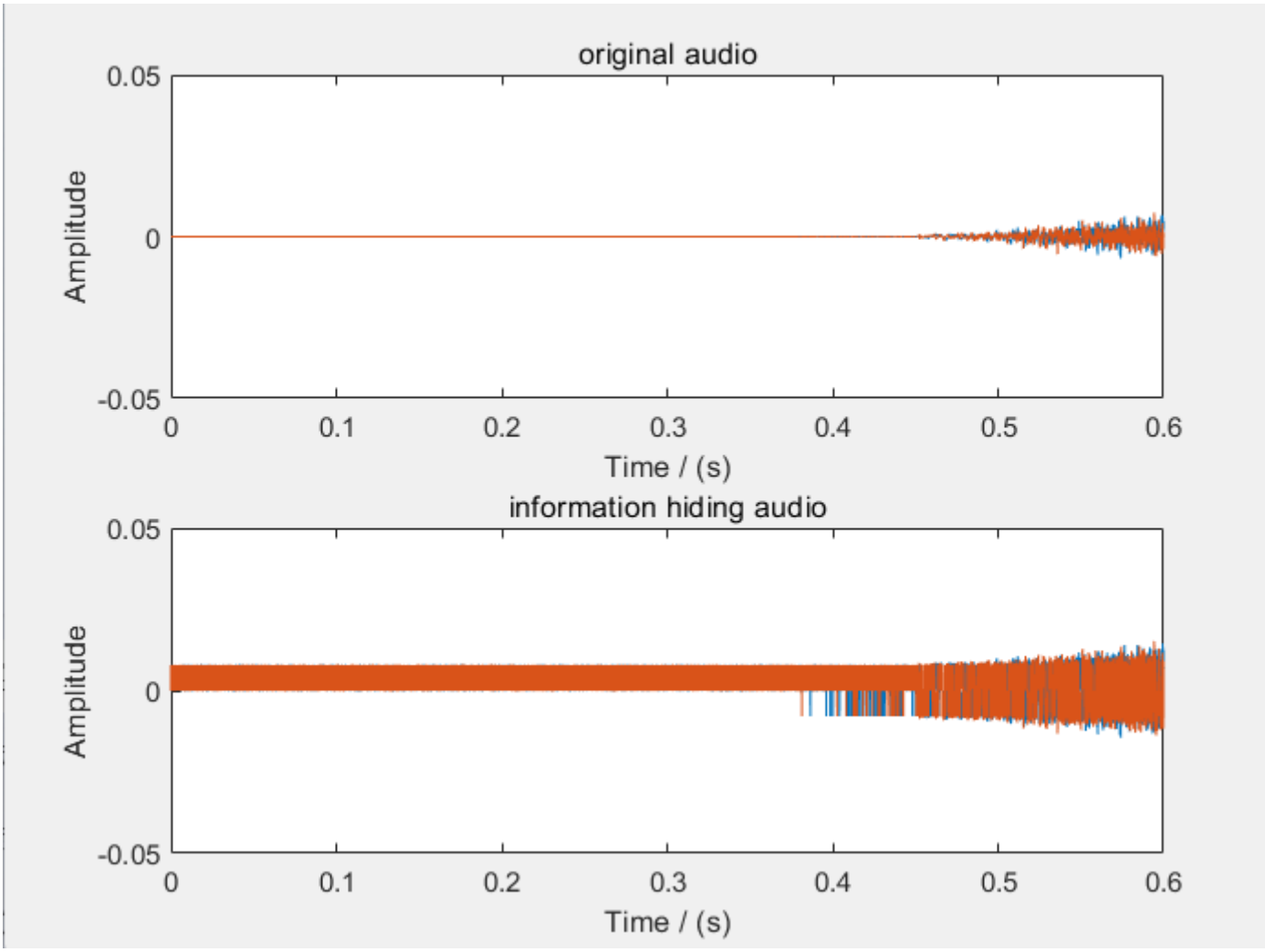
7. 绘制波形图对比：分别读取原始音频文件 `orign.wav` 和新生成的音频文件 `secret1.wav` 的音频数据和采样率，创建两个图形窗口，在每个窗口中分别绘制原始音频和隐藏信息后的音频的整体波形图以及局部细节波形图，并设置相应的坐标轴范围、标签和标题。

结果如下

可以看到隐藏秘密信息后的音频图像与之前的差异不明显



放大局部之后可以发现显著差异



三、秘密信息提取的实现

上面实验中我们已经通过最低有效位（**Least Significant Bit, LSB**）的方式，从一段音频文件中提取出之前隐藏的秘密信息。具体步骤如下：

首先，使用 `fopen` 和 `fread` 函数读取已经嵌入了秘密信息的音频文件 `secret.wav`。该音频文件是前一步实验中，通过将一段随机二进制数据嵌入原始音频的 LSB 中生成的。

读取完成后，所有音频数据以 `uint8` （8位无符号整数）格式保存在向量 `m` 中。根据 WAV 文件格式的特点，文件前 44 字节通常为文件头（Header），在本实验中，为了更普适地考虑音频格式中可能包含的额外信息，我们从第 **157 个字节开始**提取隐藏信息，避开了音频文件头对数据的干扰。

接着，利用 MATLAB 中的 `bitget` 函数，从 `m` 中的每一个字节提取出 **最低有效位**（第1位），并将其依次保存在向量 `w` 中，作为我们成功提取得到的隐藏信息序列。

完成提取后，为了评估隐藏信息的完整性与正确性，我们将 `w` 与前一阶段嵌入时所使用的原始数据 `d` 进行逐位比较。通过计算两者不相等的位数占总位数的比例，得出提取过程中的**误码率（Bit Error Rate, BER）**。

最后，将误码率通过 `fprintf` 函数打印输出，以便在命令行界面清晰地查看提取效果。误码率越低，说明隐藏信息在音频中传播和提取过程中受到的干扰越小，信息隐藏算法的稳定性和保真度越高。

```
%提取信息
fid = fopen('secret.wav', 'r');  %读取含有隐藏信息的音频文件
oa = fread(fid, inf, 'uint8');
m = oa;  %含有隐藏信息的音频采样数据赋值给 m
n = length(oa) - 156;
%利用 bitget 函数，将 m 从第 157 个元素开始的每一位的最低位提取出来，存储在w中，即为秘密数据
for i = 1 : n
    w(i) = bitget(m(156 + i), 1);
end
fclose(fid);
rate = length(find(w ~= d)) / length(d);  %将提取出的w和原信息d比较，计算误码率

% 打印误码率到命令行
fprintf('信息提取完成，误码率为：%.4f\n', rate);
```

最终结果如下：

```
>> LSB
信息提取完成，误码率为：0.0000
```

造成误码率为 0 的主要原因可能包括：

无压缩数据格式

- 本实验使用的是 `.wav` 格式音频，它是一种**无损压缩或原始PCM格式**，不会破坏底层二进制数据；
- 因此，即使我们修改了每个采样点的最低有效位，这些修改也**能被完好保留并准确提取**。

嵌入与提取算法对称

- 实验中，我们用 `bitset` 将信息嵌入到每个字节的最低位；
- 再用 `bitget` 提取这些最低位；
- 两者在操作逻辑上是一一对应的，**无信息丢失或格式错位**。

实验总结

在本次音频信息隐藏实验中，我们不仅掌握了基本的隐藏与提取操作流程，更深刻地体会到了**音频信息隐藏技术的研究意义及其广泛的应用潜力**。通过本实验，我对数字水印技术在音频领域中的实际应用有了更全面的认识，特别是在**身份验证、版权保护、数字版权管理**等方面展现出的重要价值，使我意识到这项技术在现代信息安全中的重要地位。

在实验过程中，我们借助 UltraEdit 等工具对 WAV 文件的格式结构进行了分析，深入了解了音频文件的数据组成，包括文件头（Header）和采样数据区等关键部分。通过对字节层面数据的观察，进一步

明确了信息隐藏的可能位置，从而掌握了**最低有效位法（LSB）的原理与操作方式，并初步探索了相位隐藏法、离散傅立叶变换（DFT）等高级音频隐写方法。这使我不仅停留在使用某一种方法，而是对音频隐写术的多种技术路径有了系统性了解**，并能初步对其**隐蔽性、鲁棒性、信息容量等性能指标进行比较分析**。

在 MATLAB 编程实践中，我们利用 `fopen`、`fread`、`bitset`、`bitget`、`fwrite` 等函数对音频数据进行了读写与位操作，完整实现了秘密信息的嵌入与提取流程。实验结果表明，所提取的隐藏信息与原始信息完全一致，误码率为 0，验证了 LSB 方法在无损音频文件中具有良好的可行性与精度。这一过程不仅加深了我们对 MATLAB 音频处理功能的理解，也显著提升了我在**数字信号处理、信息安全与算法实现方面的动手能力**。

总的来说，这次实验让我们从理论与实践两个层面，全面学习了音频信息隐藏的基本方法与实际应用流程。在未来的学习中，我们希望进一步探索更加复杂的音频水印算法，提升系统的抗干扰能力与信息安全性。同时，我们也意识到信息隐藏不仅仅是信息技术的“附属功能”，它更是一门融合了通信、密码学、信号处理等多学科交叉的核心技术，在数字时代具有不可替代的战略意义。